L·I·C·E·O
MATOVELLE



Nombre del proyecto:

La Programación

Liceo Matovelle
Bogotá, D.C., 11 Noviembre 2026

L·I·C·E·O
MATOVELLE



Nombre del proyecto:

La Programación

Integrante:

María Camila Jerez Acero

Tutora:

María Mercedes Pérez Jutinico

Liceo Matovelle
Bogotá, D.C., 11 Noviembre 2026

1. AGRADECIMIENTOS

Hoy quiero dar gracias a Dios por acompañarme en este camino, darme sabiduría y permitirme cumplir esta meta tan especial.

A mi familia, gracias por su amor, paciencia, apoyo y por estar siempre a mi lado, motivándome a aprender y a dar lo mejor de mí. Gracias por creer en mí y ayudarme a hacer realidad este sueño.

A mis profesores, gracias por enseñarme con cariño, por compartir sus conocimientos y guiarme durante este proceso.

Hoy me siento muy feliz y orgullosa de presentar mi proyecto final, porque aprendí que con dedicación, esfuerzo y amor podemos alcanzar nuestras metas.

¡Gracias de todo corazón por ser parte de este logro tan importante en mi vida!

2. DEDICATORIA

Dedico este logro a Dios, por guiar siempre mis pasos, y a mi familia, por ser mi mayor inspiración, por llenarme de amor y apoyarme en cada momento. Gracias por creer en mí, tomar mi mano y enseñarme que, con amor y esfuerzo, puedo alcanzar mis sueños.

Este logro también es de ustedes, porque con su amor me ayudan a crecer y a ser cada día mejor. ¡Los amo con todo mi corazón!

3. NOTAS DE ACEPTACIÓN

Presidente del jurado

Jurado

Jurado

TABLA DE CONTENIDO

1 AGRADECIMIENTOS

2 DEDICATORIA

3 NOTAS DE ACEPTACIÓN

4 RESUMEN

5 INTRODUCCIÓN

6 GLOSARIO

7 PREGUNTA PROBLEMA

8 OBJETIVOS
- 8.1 Objetivo General
- 8.2 Objetivos Específicos

9 JUSTIFICACIÓN

10 HIPÓTESIS

11 ANTECEDENTES

12 MARCO TEÓRICO
- 12.1 Definición
- 12.2 Historia de la programación
- 12.3 ¿Qué es un algoritmo?
- 12.4 Lenguajes de programación
- 12.5 Pensamiento computacional
- 12.6 Programación por bloques
- 12.7 Elementos básicos de la programación
- 12.8 Aplicaciones de la programación
- 12.9 La programación en la educación
- 12.10 La programación y la robótica
- 12.11 Personas importantes

13 MARCO CONCEPTUAL
- 13.1 Desarrollo del Proyecto
- 13.2 Planteamiento del proyecto
- 13.3 Materiales
- 13.4 Armado del piano
- 13.5 Programación en Scratch

13.6 Pruebas y funcionamiento
13.7 Relación con el pensamiento lógico

14 CONCLUSIONES

15 REFERENCIAS BIBLIOGRÁFICAS

4. RESUMEN

Este proyecto investiga qué es la programación, cómo ha evolucionado desde la tarjeta perforada hasta la inteligencia artificial y cuáles son sus elementos básicos: los algoritmos, los lenguajes de programación, el pensamiento computacional y los diagramas que ayudan a planificar el código. También se revisan sus aplicaciones en la vida cotidiana, su importancia en la educación y su relación con la robótica. Como aplicación práctica, se diseñó y desarrolló un piano de cuatro teclas con Makey Makey y programación por bloques en Scratch, con el fin de responder cómo ayuda la programación a desarrollar el pensamiento lógico y la resolución de problemas en estudiantes de grado quinto.

5. INTRODUCCIÓN

En la actualidad la tecnología es una parte fundamental en la vida cotidiana, estando presente en muchas de las actividades y espacios que las personas habitan. Por esta razón, es importante que los niños comiencen a informarse y conocer el mundo de la tecnología a través de la programación desde temprana edad, ya que esto permite desarrollar habilidades como la creatividad, pensamiento lógico, resolución de problemas y trabajo en equipo.

En este proyecto se busca visibilizar esta disciplina en niños de básica primaria, para que puedan adquirir las bases fundamentales de la programación y motivarlos a que sigan explorando y aprendiendo de esta rama, con el fin de que los niños puedan prepararse para los retos y oportunidades del futuro.

6. GLOSARIO

COBOL: Lenguaje de programación de alto nivel compilado y de propósito general. Diseñado para el procesamiento de datos comerciales, financieros y administrativos.

Compilador: Traduce el código fuente escrito en un lenguaje de programación de alto nivel (C++, Java) a lenguaje de máquina o binario (ceros y unos).

Diagrama de flujo: Representación gráfica de los pasos de un algoritmo, ordenados con figuras y flechas. Cada figura del diagrama tiene un significado:

- **Óvalo:** indica el inicio y el final del proceso.
- **Rectángulo:** representa una acción o instrucción que se debe realizar.
- **Rombo:** representa una decisión, con respuestas como «sí» o «no».
- **Flecha:** une las figuras e indica el orden en que se siguen los pasos.

FORTRAN: Lenguaje de programación de alto nivel compilado y de propósito general. Diseñado para cálculos numéricos avanzados y física computacional.

Inteligencia artificial generativa: Es un tipo capaz de crear contenido nuevo y original, a partir de las peticiones o instrucciones que escribe un usuario (*prompt*).

Inteligencia artificial predictiva: Usa datos del pasado, estadísticas y aprendizaje automático para encontrar patrones y predecir qué pasará o qué tendencias ocurrirán en el futuro.

Lenguaje ensamblador: Es un tipo de programación de bajo nivel que usa símbolos y palabras clave simples, para dar órdenes directas al procesador de una computadora.

Lenguajes de programación de alto nivel: Es aquel que usa palabras y reglas muy parecidas al lenguaje humano, ayudando a que las personas puedan leer, escribir y entender el código de forma muy sencilla.

Pseudocódigo: Estructuras de programación para planificar la lógica de un software sin depender de una sintaxis rígida.

Sistemas operativos: Programa principal de una computadora o celular que ayuda a que funcionen las aplicaciones. Actúa como un puente entre el usuario y la máquina.

Software: Conjunto de instrucciones, programas y datos intangibles que le indican a una computadora o dispositivo cómo debe funcionar.

Tarjetas perforadas: Lámina de cartulina que contiene datos o programas representados mediante la presencia (valor 1) y la ausencia (valor 0) de agujeros en posiciones predeterminadas.

7. PREGUNTA PROBLEMA

¿Cómo ayuda la programación a desarrollar el pensamiento lógico y la resolución de problemas en los estudiantes de grado quinto?

8. OBJETIVOS

8.1 Objetivo General

Investigar la importancia de la programación y desarrollar un proyecto que permita aplicar los conocimientos adquiridos.

8.2 Objetivos Específicos

1. Comprender qué es la programación y cómo ha evolucionado.
2. Identificar los elementos básicos de un programa.
3. Reconocer las aplicaciones de la programación en la vida cotidiana.
4. Aprender a utilizar una plataforma de programación por bloques.
5. Diseñar y desarrollar un proyecto de programación.

9. JUSTIFICACIÓN

En este proyecto surge la necesidad de acercar a los estudiantes de grado quinto al mundo de la programación, una habilidad cada vez más relevante en el siglo XXI. Los estudiantes no solo aprenderán conceptos básicos de lógica y secuencias, sino que también desarrollarán competencias como el pensamiento computacional, la resolución de problemas y el trabajo en equipo.

La programación en esta etapa escolar permite que los niños comprendan cómo funcionan las tecnologías que usan a diario, fomentando la creatividad y la autonomía. Además, al plantear retos cercanos a su realidad, se motiva su interés y se fortalece su confianza para enfrentar desafíos.

10. HIPÓTESIS

Si los estudiantes aprenden los fundamentos de la programación mediante actividades prácticas, entonces fortalecerán su pensamiento lógico, su creatividad y su capacidad para resolver problemas.

11. ANTECEDENTES

Según Morales et al. (2020), la programación LOGO es de carácter textual, pero también está la programación visual; para Recarde et al. (2018), la programación visual se caracteriza por formar algoritmos por medio de bloques, los cuales son arrastrados y organizados de tal forma que constituyen secuencias lógicas y forman un determinado programa. Aprender a programar desde la infancia tiene unas ventajas significativas, entre las cuales están: desarrolla el pensamiento lógico-matemático, los estudiantes pasan de ser simples consumidores de tecnología a crear sus propios programas, fomenta el trabajo en equipo y los niños aprenden a comunicarse con la computadora, aspecto fundamental en las competencias del siglo veintiuno, lo cual a su vez permite que sean más creativos, desarrollen problemas de forma lúdica y estructurada, entre otros aspectos.

Colombia es uno de los países en los cuales la transformación en la política pública y en el aspecto curricular, según Riesco et al. (2014), está relegada a mantener consumidores de tecnología, en donde los estudiantes conocen aplicaciones y las usan, más no son prosumidores. Para Manrique et al. (2020), el cambio está en la innovación curricular y para esto propone dos ejes articuladores, enfocados en la programación y la robótica. También cabe resaltar los esfuerzos del Ministerio de Educación y el Ministerio de Tecnologías de la Información y Comunicaciones en convenio con el British Council para capacitar a los docentes del sector público en cursos virtuales de corta duración, en la programación para niños por medio del editor Makecode; desde el año 2019, los docentes replican con sus estudiantes sus aprendizajes (Cerón, 2021).

La enseñanza de la programación en la educación básica ha tomado fuerza en las últimas décadas, porque se reconoce que el pensamiento computacional es una habilidad transversal y no exclusiva de quienes serán programadores. Wing (2006) lo define como un conjunto de procesos de pensamiento que permiten resolver problemas, diseñar sistemas y comprender el comportamiento humano apoyándose en conceptos fundamentales de la informática; además, sostiene que esta habilidad debería desarrollarse desde la infancia, igual que la lectura, la escritura y la aritmética.

En el ámbito internacional, distintos países han incorporado la programación en sus cu-

rrículos escolares desde edades tempranas, apoyándose en entornos visuales por bloques. Resnick et al. (2009), creadores de Scratch en el MIT Media Lab, plantean que la programación debe aprenderse como una actividad creativa y social: los niños construyen proyectos significativos, los comparten y aprenden de los demás. Estas experiencias evidencian que los estudiantes de primaria pueden comprender conceptos como secuencia, repetición y condición cuando se presentan con ejemplos cercanos y con un propósito claro.

Por su parte, Papert (1980), con su propuesta del construccionismo, sostuvo que los niños aprenden mejor cuando construyen objetos que pueden compartir y reflexionar sobre ellos; su trabajo con el lenguaje Logo sentó las bases de gran parte de las herramientas actuales de programación escolar. En la misma línea, Bers (2018) señala que la programación puede introducirse desde los primeros grados si se emplean materiales concretos, actividades lúdicas y un acompañamiento adecuado por parte del docente.

12. MARCO TEÓRICO

12.1 Definición

La programación es el proceso formal de diseñar, codificar, depurar y mantener el código fuente de un programa, mediante el cual se traducen algoritmos lógicos en un lenguaje comprensible para una computadora, con el objetivo de resolver problemas o ejecutar tareas específicas de manera automatizada.

12.2 Historia de la programación

La programación tuvo su origen en la década de 1800, cuando Joseph Marie Jacquard creó las tarjetas perforadas. Posteriormente, el matemático Charles Babbage intentó construir una máquina programable para hacer cualquier tipo de cálculo, y Ada Lovelace publicó el primer algoritmo que debía seguir esta máquina para poder ser ejecutada. Sin embargo, nunca fue probado, porque la máquina nunca fue construida.

Un siglo después, en medio de la Segunda Guerra Mundial y como una necesidad de sobrevivir, Alan Turing creó la máquina Bombe, que desencriptaba y descifraba los mensajes del ejército alemán para adelantarse a sus estrategias y poder derrotarlos. Actualmente, Alan Turing es considerado el padre de la Ciencia de la Computación y pionero en la Inteligencia Artificial.

En 1946, se creó la ENIAC (*Electronic Numerical Integrator and Computer*), siendo esta la primera computadora electrónica con un propósito general en el mundo. Fue diseñada por John Mauchly y John Presper Eckert en la Universidad de Pensilvania, financiada por el Ejército de Estados Unidos, con el propósito de calcular tablas de tiro de artillería. Además, se considera una máquina innovadora, ya que era reprogramable para resolver una gran variedad de problemas numéricos en física y matemáticas.

En 1947, la británica Kathleen Booth creó el lenguaje ensamblador para la computadora ARC. En 1950, el uso de lenguajes ensambladores se popularizó expandiéndose a otras computadoras. Al mismo tiempo se desarrollaban lenguajes de programación de alto nivel, como Fortran

y COBOL.

En 1970, se desarrollaron lenguajes como C y Pascal, que abrieron camino para el desarrollo de software y sistemas operativos. En 1980, se desarrolló C++, que introdujo la programación orientada a objetos. En 1989, se creó Python por Guido van Rossum; su objetivo era crear un lenguaje de programación que fuera fácil de leer y que favoreciera la productividad.

En 1990, se creó la World Wide Web, impulsando el uso de lenguajes como HTML y JavaScript para el desarrollo de páginas web.

En la actualidad, la programación se centra en el desarrollo de la Inteligencia Artificial, tanto generativa como predictiva, el análisis de datos, el desarrollo de software, la ciberseguridad, la computación en la nube y la computación cuántica.

12.3 ¿Qué es un algoritmo?

Un algoritmo es un conjunto de instrucciones ordenadas para resolver un problema, realizar un cálculo o desarrollar una tarea. En la programación un algoritmo supone el paso previo a escribir el código. Este le va a dar el impulso a la computadora, celular, máquina, etc., a seguir un paso a paso. También va a permitir la automatización y procesamiento avanzado de instrucciones. Desde una perspectiva técnica, en programación un algoritmo es la descripción lógica y estructurada de un proceso que toma datos de entrada, los transforma mediante operaciones específicas y devuelve una salida concreta. Esta salida puede ser un valor numérico, un texto, una acción en pantalla o incluso la activación de otro proceso dentro de un sistema.

12.4 Lenguajes de programación

Los lenguajes de programación se categorizan mediante diferentes criterios:

Lenguajes de bajo nivel: es el que da las instrucciones con un lenguaje binario y ensamblador que permiten escribir un código eficiente pero complejo. Siendo este un tipo de lenguaje de programación poco entendible por los humanos. Ej.: Assembly, microprogramación y código máquina.

Lenguajes de alto nivel: como Python, JavaScript, Java y C++ (se considera de nivel

medio); estos se enfocan en que el lenguaje de computadora sea más fácil de entender para los humanos, pero contienen instrucciones que la computadora sabe interpretar.

Tabla 1

Diferencias entre los lenguajes de programación

Categoría	**Lenguaje de programación**	**Sintaxis (¿cómo se escribe el programa?)**	**¿Para qué se usa?**	**¿Quién controla la memoria?**	**¿Qué tan fácil es de leer?**
Alto nivel	Python	Sin punto y coma, sin llaves, indentación obligatoria, comentarios, sensible a mayúsculas	IA, *machine learning*, ciencia y análisis de datos	El gestor de memoria de Python, a través del intérprete CPython	Es muy fácil, pues su creador lo diseñó específicamente para esto
Bajo nivel	Assembly	Etiqueta, comentario, operandos	Desarrollo de sistemas operativos, controladores de dispositivos	El programador	Muy difícil
Nivel medio	C++	Punto y coma obligatorio, bloques con llaves, tipado estricto, funciones	Videojuegos de alto rendimiento, bases de datos, aplicaciones de diseño	El programador, con `new` y `delete`	Moderado a difícil
Alto nivel	Java	Orientación a objetos, sensible a mayúsculas, el nombre del archivo importa, tipado estricto	Aplicaciones empresariales y corporativas, aplicaciones móviles Android	Se maneja de forma automática con *Garbage Collector*	Fácil a moderado
Alto nivel	JavaScript	Punto y coma opcional, tipado dinámico y flexible, variables con alcance, integración nativa	Desarrollo *frontend*, desarrollo *backend*, videojuegos y navegadores	Forma automática mediante un *Garbage Collector*	Fácil
Alto nivel	Scratch	Movimiento, apariencia, eventos, control, operadores	Educación, creación de videojuegos 2D, animaciones, introducción a la robótica	Forma automática y oculta	Extremadamente fácil
Bajo nivel	Código máquina	Campos fijos de bits	Ejecución directa en el hardware, destino de compilación	El programador o el compilador maneja la memoria	Casi imposible de leer: son cadenas de unos y ceros o su representación en sistema hexadecimal
Bajo nivel	Micro-programación	Microcódigo	Diseñar e implementar la Unidad de Control de un microprocesador	No existe el concepto de manejo de memoria	Extremadamente difícil

12.5 Pensamiento computacional

Es el proceso de pensamiento que permite formular o resolver problemas del mundo real, haciendo uso de habilidades y técnicas, como las secuencias e instrucciones ordenadas (algoritmos), para llegar a la solución.

Implica identificar, representar, organizar y analizar lógicamente la información, implementando posibles soluciones con el objetivo de lograr la combinación más efectiva y eficiente entre pasos y recursos.

12.6 Programación por bloques

La programación por bloques es un método para crear *scripts* y aplicaciones sin necesidad de escribir texto manualmente. En lugar de teclear de forma tediosa, el usuario dispone de elementos gráficos que recuerdan a bloques o piezas de *puzzle* virtuales. Cada uno de esos elementos representa un comando, una instrucción o una estructura lógica concreta.

12.7 Elementos básicos de la programación

Dentro de los elementos de programación más utilizados para llevar a cabo una buena práctica de programación están, por ejemplo, los algoritmos, el pseudocódigo, los diagramas de flujo, los diagramas de bloques, los mapas de entidad-relación y los diagramas de flujo de datos.

12.8 Aplicaciones de la programación

La programación se usa en:

- Desarrollo de software y aplicaciones
    - Páginas web y plataformas digitales
    - Aplicaciones
    - Sistemas operativos
- Inteligencia artificial y datos
    - *Machine learning*

    - Análisis de *big data*
- Entretenimiento y videojuegos
    - Videojuegos
    - Efectos especiales y animación
- Ciencias, salud y finanzas
    - Medicina y salud
    - Finanzas y banca

12.8.1 La programación en la vida cotidiana

La programación está presente en muchas actividades diarias. En celulares y computadores se usa en aplicaciones, juegos y redes sociales; en el hogar, lavadoras, televisores y asistentes de voz siguen instrucciones programadas; en la ciudad, semáforos, cajeros automáticos y GPS funcionan gracias a ella; y en la escuela y la salud, mediante plataformas educativas y citas médicas en línea.

12.9 La programación en la educación

La programación en la educación es importante porque ofrece un marco de trabajo para facilitar el aprendizaje. También facilita el desarrollo de habilidades críticas y analíticas en los estudiantes, favoreciendo su capacidad para la resolución de problemas y pensar de manera lógica.

En el aula, la programación desarrolla el pensamiento lógico y computacional desde pequeños, fortalece la creatividad y la capacidad de resolver problemas, enseña a probar, corregir errores y trabajar en equipo, y prepara a los estudiantes para un mundo digital, pues es una habilidad para el futuro.

12.10 La programación y la robótica

La programación robótica es un tipo de procedimiento que nos ayuda a preparar ciertos dispositivos para que, en base a una serie de factores, se puedan comportar de una forma u otra.

Las instrucciones programadas determinan cómo responde el dispositivo ante esos factores. Un ejemplo de robótica aplicada como forma de educación es el programa Escuela de ROBOTS-

chool, según su descripción pública; este contenido se refiere a ese programa y no presupone que todos sus cursos tengan la misma organización.

12.11 Personas importantes

1. **Charles Babbage:** se considera el padre de los computadores digitales, pues construyó un modelo de calculadora mecánica.
2. **Ada Lovelace:** creó el primer algoritmo para ser procesado por una máquina.
3. **Alan Turing:** fue el creador de la máquina electromecánica precursora de los computadores modernos. Se considera el padre de la ciencia de la computación y pionero de la Inteligencia Artificial.
4. **Grace Hopper:** diseñó el lenguaje COBOL y el primer compilador.
5. **Margaret Hamilton:** diseñó el software guía para llegar a la Luna.
6. **Dennis Ritchie:** inventó el lenguaje C y el sistema Unix.
7. **Linus Torvalds:** desarrolló el núcleo de Linux y el control de versiones Git.
8. **Guido van Rossum:** creó el lenguaje de programación Python.
9. **Tim Berners-Lee:** inventó la World Wide Web (WWW).

13. MARCO CONCEPTUAL

13.1 Desarrollo del Proyecto

Para cumplir el objetivo de diseñar y desarrollar un proyecto de programación por bloques, se construyó un piano de cuatro teclas que combina un componente físico (la placa Makey Makey y teclas de papel de aluminio) con un componente de programación (Scratch). Con este trabajo se aplicó lo aprendido sobre algoritmos, secuencias, eventos y pensamiento computacional.

En este proyecto se uso mblock como lenguaje de programacion por bloques, ya que es un software de programación visual que permite a los estudiantes crear programas mediante la manipulación de bloques gráficos. Este enfoque facilita la comprensión de conceptos de programación y fomenta la creatividad y el pensamiento lógico. Mblock nacio como una plataforma educativa basada en Scratch, diseñada para enseñar programación y robótica a estudiantes de todas las edades. Su interfaz intuitiva y amigable permite a los usuarios arrastrar y soltar bloques de código para crear programas interactivos, juegos y proyectos de robótica.

Adicionalmente, se utilizó la placa Makey Makey, un dispositivo que convierte objetos cotidianos en teclas de entrada para computadoras. Esta placa permite a los estudiantes interactuar con su entorno físico y digital, fomentando la creatividad y la experimentación.

Finalmente, se desarrolló una interfaz web como complemento del piano físico. Para construirla se utilizó JavaScript, un lenguaje de programación que permite crear páginas web interactivas. Como punto de partida se analizó el código diseñado en mBlock con Python y, paso a paso, se tradujo esa misma idea al nuevo lenguaje. De esta manera se logró que la música y la lógica original del proyecto funcionaran en una página web moderna, sin perder la experiencia de tocar el piano de forma física.

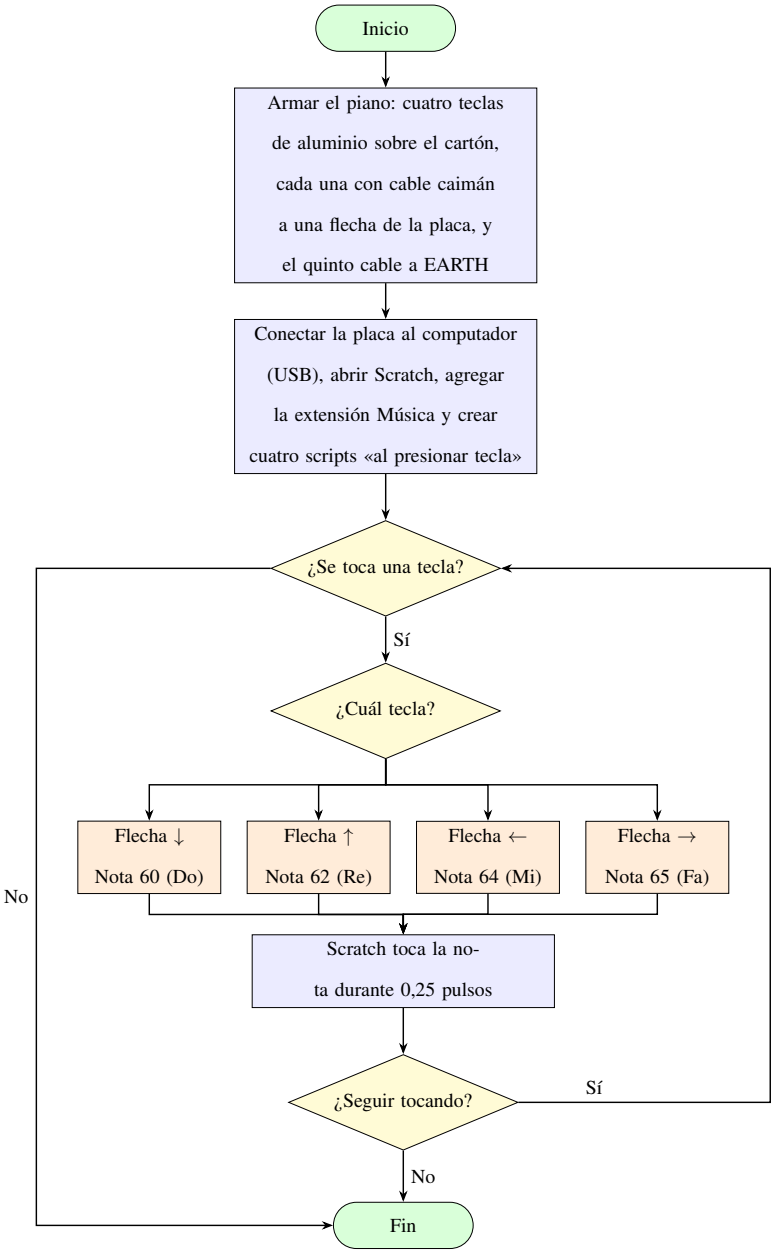
13.2 Planteamiento del proyecto

La idea consiste en construir un piano de cuatro teclas en el que cada tecla toque una nota musical. La placa Makey Makey permite que objetos conductores, como el papel de aluminio, funcionen como teclas del computador, y Scratch recibe esas pulsaciones y ejecuta el sonido programado. El proyecto busca responder la pregunta problema: cómo ayuda la programación a desarrollar el pensamiento lógico y la resolución de problemas en estudiantes de grado quinto.

Para dar respuesta a esta pregunta, el proyecto se planificó antes de construirse. La elaboración de un diagrama de flujo es importante para quienes desarrollan programas, ya que es una herramienta que ayuda a diseñar las soluciones de forma más organizada antes de escribir el código. Además, permite seguir el recorrido del programa paso a paso, lo que facilita identificar errores o fallas. Por esta razón, en la Figura 1 se presenta el diagrama de flujo del piano, que resume en orden los pasos del proyecto: armar y conectar el piano, programar en Scratch y decidir qué nota suena según la tecla que se toca. Elaborar este diagrama exige ordenar los pasos y pensar qué ocurre en cada decisión, lo cual hace parte del pensamiento computacional y ayuda a que los estudiantes de grado quinto comprendan conceptos como secuencia y condición.

Figura 1

Diagrama de flujo del funcionamiento del piano con Makey Makey y Scratch



Nota. Diagrama elaborado por la autora para explicar el funcionamiento del piano.

13.3 Materiales

- Placa Makey Makey con su cable USB.
- Cinco cables caimán: cuatro para las entradas y uno para EARTH.

- Cartón como base del piano.
- Papel de aluminio para las teclas.
- Cinta para fijar los materiales.
- Computador con la plataforma Scratch.

Las teclas deben permanecer separadas entre sí para que cada una funcione de manera independiente.

13.4 Armado del piano

1. **Teclas:** se construyen cuatro teclas de papel de aluminio sobre el cartón, separadas entre sí.
2. **Conexión:** se une cada tecla con un cable caimán a una entrada de la placa, una flecha por tecla.
3. **EARTH:** se conecta el quinto cable a EARTH y se mantiene contacto con el metal para cerrar el circuito.

Figura 2

Armado del circuito durante la clase de robótica



Nota. Evidencia fotográfica del armado y la conexión de los cables, tomada de la presentación del trabajo.

13.5 Programación en Scratch

1. **Extensión:** se abre Scratch y se agrega la extensión Música.
2. **Scripts:** se crean cuatro *scripts* con el bloque «al presionar tecla», uno por cada flecha de la placa.

3. **Notas:** cada *script* toca una nota distinta, las notas 60, 62, 64 y 65, durante 0,25 pulsos (en la escala musical equivalen a Do, Re, Mi y Fa).

Cada *script* es un pequeño algoritmo: un evento (presionar la tecla) activa una instrucción (tocar la nota) con una duración definida. Al ser programación por bloques, no fue necesario escribir código de forma manual.

13.6 Pruebas y funcionamiento

1. **Prueba 1:** se comprueba cada nota con el teclado del computador, para verificar que los *scripts* funcionan.
2. **Prueba 2:** se tocan las teclas físicas y cada una debe sonar con su nota.

Figura 3

Programación y pruebas del piano en el computador



Nota. Evidencia fotográfica de la práctica con el computador, tomada de la presentación del trabajo.

Durante las pruebas se debe mantener la placa y el computador secos. Si una tecla no suena, se revisa la conexión del cable, que el contacto con EARTH esté cerrado y que las teclas no se toquen entre sí; este proceso de probar y corregir errores es parte del pensamiento computacional.

13.7 Relación con el pensamiento lógico

El proyecto exigió ordenar pasos, relacionar cada tecla con una instrucción, probar y corregir. De esta manera se evidencia que las actividades prácticas, como la robótica y el piano con Scratch, fortalecen el pensamiento lógico, la creatividad y la capacidad de resolver problemas, lo

cual respalda la hipótesis planteada. El trabajo se realizó con computador y cables durante la clase de robótica.

14. CONCLUSIONES

Este proyecto permitió comprender que la programación es el proceso de diseñar, codificar, depurar y mantener programas, y que ha evolucionado desde la tarjeta perforada hasta la inteligencia artificial. En ese recorrido se identificó que los elementos básicos de un programa son los algoritmos, los lenguajes de programación, el pensamiento computacional y los diagramas, como el diagrama de flujo, que ayudan a planificar el código antes de escribirlo.

Estos conceptos no se quedan en la teoría, pues la programación está presente en la vida cotidiana: en los celulares, el hogar, la ciudad, los juegos, la salud y las finanzas. Reconocer esa presencia permite entender por qué es importante que los niños se acerquen a la programación desde temprana edad.

Con esa base se diseñó y desarrolló el piano de cuatro teclas, que permitió responder la pregunta problema: la programación ayuda a desarrollar el pensamiento lógico y la resolución de problemas porque exige ordenar pasos, probar y corregir errores. Además, la programación por bloques en Scratch facilitó este aprendizaje sin necesidad de escribir código, y el proyecto aplicó lo aprendido en un trabajo concreto.

De esta manera, lo investigado respalda la hipótesis planteada: las actividades prácticas, como la robótica y el piano con Scratch, fortalecen el pensamiento lógico, la creatividad y la capacidad de resolver problemas en grado quinto. Por esta razón, los colegios deben implementar la programación desde los primeros grados, porque es una habilidad para el futuro de nuestra generación.

15. REFERENCIAS BIBLIOGRÁFICAS

Bejob. (2017, 23 de febrero). *¿Qué es la programación robótica y para qué sirve?* https://www.bejob.com/que-es-la-programacion-robotica-y-para-que-sirve-2/

Coding Giants. (2026, 8 de mayo). *¿Qué es la programación por bloques?* https://codinggiants.es/blog/que-es-la-programacion-por-bloques

Cultura. (2020, 22 de junio). *Alan Turing, el padre de la inteligencia artificial*. https://www.cultura.gob.ar/alan-turing-el-padre-de-la-inteligencia-artificial-9162/

Díaz Garcés, J. (2023, 23 de octubre). *Historia de la programación*. CIPSA. https://cipsa.net/historia-de-la-programacion/

Encyclopaedia Britannica. (s. f.). *ENIAC*. https://www.britannica.com/technology/ENIAC

Grupo Codesi. (s. f.). *Historia de Python: origen, evolución y su impacto en la programación moderna*. https://www.grupocodesi.com/blog/historia-de-python.html

Joyanes Aguilar, L. (s. f.). *Fundamentos de programación* (4.ª ed.). Internet Archive. https://archive.org/details/fundamentos-de-programacion-4ta-edicion-luis-joyanes-aguilar/page/52/mode/2up

Revista Convicciones. (s. f.). Fundación de Estudios Superiores Comfanorte. https://www.fesc.edu.co/Revistas/OJS/index.php/convicciones/article/view/1900

ROBOTSchool. (s. f.-a). *Escuela de ROBOTSchool* [Sitio web]. https://class.miel-robotschool.com/escuela/ventas.html

ROBOTSchool. (s. f.-b). *ROBOTSchool* [Sitio web]. https://robotschool.com.co/

Rodríguez Candela, U. (2026, 24 de enero). *La evolución de la programación: historia completa de los lenguajes y tecnologías*. Ulises.io. https://ulises.io/blog/evolucion-programacion-historia-2026

Wikipedia. (s. f.-a). Kathleen Booth. En *Wikipedia, la enciclopedia libre*. https://es.wikipedia.org/wiki/Kathleen_Booth

Wikipedia. (s. f.-b). Lenguaje ensamblador. En *Wikipedia, la enciclopedia libre*. https://es.wikipedia.org/wiki/Lenguaje_ensamblador

Wikipedia. (s. f.-c). Tarjeta perforada. En *Wikipedia, la enciclopedia libre*. https://es.wikipedia.org/wiki/Tarjeta_perforada